

AI Scraper fertig bauen → n8n anbindung

👤 Assignee	👤 Ana
☰ Notizen	letzter Schritt
⚡ Priorität	NOW
⚙️ Status	In Bearbeitung
☰ Topic	CODE

Hi Firas,

Here's a quick overview of the tech stack I chose for my prototype "Briefed" — a personalized AI-powered news agent:

- News Data – [NewsAPI.org](https://newsapi.org/) (free tier): Fetches headlines by keyword. Chosen for its simple REST API and free tier availability. Known limitation: returns metadata only, not full article text.
- AI Summarization – Azure AI Foundry (GPT-5.4-mini): Generates concise, personalized summaries based on user interests. Chosen because I have institutional access through Microsoft Foundry, avoiding additional API costs.
- Language & Core Logic – Python + Flask: All pipeline logic lives here — fetching, summarizing, emailing. Flask exposes a single /run endpoint for external triggering. Chosen for simplicity and full control over the AI pipeline.
- Database – SQLite: Stores user preferences (interests, email) and tracks articles already sent to avoid duplicates across daily runs. Chosen deliberately over PostgreSQL for prototype simplicity — the schema and logic are identical, and the layer is swappable.
- Email Delivery – Gmail SMTP: Sends the formatted daily briefing. Chosen for zero setup cost and direct integration with Python's built-in smtplib.

- Hosting – [Render.com](https://render.com) (free tier): Hosts the Flask app publicly so n8n can reach it. Auto-deploys from GitHub on every push.
- Scheduling & Orchestration – n8n.cloud (free tier): Triggers the pipeline daily via HTTP POST to the /run endpoint. Chosen to separate scheduling concerns from core logic cleanly.

The architecture follows a clear separation of responsibilities: n8n owns when the pipeline runs, Python owns what it does. Every component is on a free tier, the prototype is fully functional and demonstrable end to end.

Happy to walk through any part of it in more detail.

Best,

Ana